Doc. no.	P1094R2
Date:	2018-11-09
Project:	Programming Language C++
Audience:	Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Nested Inline Namespaces

Table of Contents

0. Revision History
    - Revision 2
    - Revision 1
    - Revision 0
1. Introduction
2. Stating the problem
    - Impact on the Language
    - Impact on the Library
3. Propose Solution
    - Compatibility Concerns
    - Initial Reception
    - Review in San Diego
4. Other Directions
    - Extend Existing `inline` Syntax
    - Harmonize for the Leading Namespace
    - Attributes on Nested Namespace
5. Formal Wording

Revision History

Revision 2

Updated proposed wording after initial feedback from Jens Mauer.

Recorded the poll results of review in San Diego.

Revised all the examples to use the more realistic example of the inline namespace `parallelism_v2` consistently through the paper, rather than just in the motivating example.

Revision 1

Updated alternative considerations of where `inline` keyword may appear, following review at Rapperswil 2018.

Revision 0

Original version of the paper for the 2017 post-Albuquerque mailing.

1 Introduction

Inline namespaces are not supported in nested namespace definitions, and it is surprising that the "obvious" syntax is not supported.

2 Stating the problem

While working on the header synopses for several TSes, it is apparent how convenient the nested namespace feature is. However, it is equally surprising that the inline versioning namespace cannot be similarly declared. For example, in the Concurrency TS v2 it would be nice to specify the `<experimental/execution>` synopsis as:

```
namespace std::experimental::inline parallelism_v2::execution {
  // 5.7, Unsequenced execution policy
  class unsequenced_policy;

  // 5.8, Vector execution policy
  class vector_policy;

  // 5.10, execution policy objects
  inline constexpr sequenced_policy seq{ unspecified };
  inline constexpr parallel_policy par{ unspecified };
}
```

Instead, we must open and close namespaces three separate times, essentially rendering the nested namespace feature useless in this case:

```
namespace std::experimental {
inline namespace parallelism_v2 {
namespace execution {
  // 5.7, Unsequenced execution policy
  class unsequenced_policy;

  // 5.8, Vector execution policy
  class vector_policy;

  // 5.10, execution policy objects
  inline constexpr sequenced_policy seq{ unspecified };
  inline constexpr parallel_policy par{ unspecified };
}
}
}
```

2.1 Impact on the Language

The concern raised is an inconvenient embarrassment, rather than a fundamental flaw. It would be nice to see this fixed, and hopefully should be possible with a minimal investment of time by the committee. If it is not seen as immediately useful, it is probably not worth further committee time to discuss alternative designs.

2.2 Impact on the Library

This is a pure language extension that cannot have any breaking impact on existing standard library wording. It does open a possibility for slightly cleaner specification of some standard library headers if LWG want to adopt this feature, but that most applies to TS specifications. The only use of inline namespaces in the standard library of ISO 14882 is to declare user defined literals, and it is not clear that there is an improvement in how those particular namespaces might be opened using this facility. There are no library wording udpates as part of this proposal.

3 Propose Solution

The proposed solution is fairly simple: allow the `inline` keyword to optionally precede the identifier naming a namespace at each step of a nested namespace declaration. It is believed that editing the grammar is a sufficient

change, as the existing text would be interpreted as having the desired meaning after this change.

3.1 Compatibility Concerns

The proposed solution is a pure extension where the new syntax was not valid before, and could not show up even in a SFINAE context. There are not expected to be any backwards compatibility concerns. As such, there is nothing to add to the compatibility annex of the standard (Annex C).

A fine-grained feature macro is not warranted in this case, as use of the macro to enable the convenience feature would be more work than the convenience returned. The regular `__cplusplus` macro should suffice for those keen to create a codebase anticipating a future cleanup that can assume this feature once support for older dialects (by that user) is dropped.

3.2 Initial Reception

This proposal was briefly presented to EWG at the end of the Rapperswil 2018 meeting, and polling in the room was broadly in favor, although it would like to see it again, with a little feedback, before sending to Core:

SF F N A SA
9 7 1 0 0

3.3 Review in San Diego

Two polls were taken in San Diego, which resulted in sending the proposal to Core to review for C++20.

1. Do we want to have support for inline namespaces in nested namespace definitions?
   SF F N A SA
   3 9 8 4 0
2. Do we want to allow inline in the leading position when declaring namespaces?
   SF F N A SA
   0 2 13 4 2

4 Other Directions

A few options were considered and rejected by the author of this proposal during the design. A quick summary of these other directions follows.

4.1 Extend Existing `inline` Syntax

Given the existing syntax does not support an `inline` *before* the `namespace` keyword for a nested namespace declaration, which is the position required in the grammar for a non-nested namespace declaration, careful consideration was given to supporting this as an option too, if for no other reason than the notion of consistency.

```
inline namespace std::experimental::parallelism_v2;  // not immediately to reader,
                                                     // is std or parallelism_v2 inline?
```

Which namespace is intended to be inline in such cases? The trailing namespace? The leading namespace? All of the namespaces? Just the trailing namespace? Requiring the `inline` keyword to precede the namespace that it inlines cleanly resolves such concerns, so there should be no other syntax supported to introduce such confusion, along with endless style-guide debates.

4.2 Harmonize for the Leading Namespace

The other side of the question is whether a top-level namespace can be declared as inline with this new syntax, by simply moving `inline` to the other side of the `namespace` keyword.

```
inline namespace parallelism_v2;  // legal today, and unambiguous to reader
namespace inline parallelism_v2;  // should this be equivalent?
```

This idea is rejected as potentially confusing once nested namespaces are involved.

```
namespace inline std::experimental::parallelism_v2;  // immediately to reader?
                                                     // std or parallelism_v2 inline?
```

This has all the same concerns of ambiguity to the reader that are raised in (4.1) above, although it is a little clearer what the grammar must mean in this case, if we wanted to apply such a rule. Instead, we require that the leading namespace cannot be inline, and must be separately opened with the old syntax. Given a quick polling of folks offline, we are not aware of any motivation to support an inline top-level namespace, so prefer to keep an unambiguously read grammar than support this terse syntax for a questionable use case.

A second concern is that the grammar for the first (current) form allows for annotating namespaces with attributes, whereas the grammar for nested namespace declarations, used by the second form, does not - see (4.3) below.

A minor concern is that introducing a second way to express exactly the same semantic that we can express today invites as many awkward questions about which to choose as the consistency problem it is intended to solve.

This point came up quickly at the end of the Rapperswil discussion, and the counterpoint was not clearly presented. The quick poll in the room was in favor of making the change to harmonize, so the wording change to effect that change is included below, in the event that EWG is strongly in favor of making this change. It involves adding one additional optional `inline` into the grammar.

enclosing-namespace-specifier:
    inline$_{opt}$ *identifier*
    enclosing-namespace-specifier :: **inline**$_{opt}$ *identifier*

4.3 Attributes on Nested Namespace

It was also observed, while drafting this paper, that nested namespaces do not support attributes. That was deemed a separate issue beyond the scope of this paper, and is not a topic the author is motivated to solve himself.

5 Formal Wording

Make the following changes to the specified working paper:

9.7.1 Namespace definition [namespace.def]

namespace-name:
    identifier
    namespace-alias

namespace-definition:
    named-namespace-definition
    unnamed-namespace-definition
    nested-namespace-definition

named-namespace-definition:
    inline$_{opt}$ **namespace** *attribute-specifier-seq*$_{opt}$ *identifier* { *namespace-body* }

unnamed-namespace-definition:
    inline$_{opt}$ **namespace** *attribute-specifier-seq*$_{opt}$ { *namespace-body* }

nested-namespace-definition:
    namespace *enclosing-namespace-specifier* :: **inline**$_{opt}$ *identifier* { *namespace-body* }

enclosing-namespace-specifier:
    identifier
    enclosing-namespace-specifier :: **inline**$_{opt}$ *identifier*

namespace-body :
    declaration-seq$_{opt}$

9. A *nested-namespace-definition* with an *enclosing-namespace-specifier* `E`, *identifier* `I` and *namespace-body* `B` is equivalent to

```
namespace E { inline_opt namespace I { B } }
```

where the optional `inline` is present if and only if the *identifier* `I` is preceded by `inline`.
[*Example:*

```
namespace A::inline B::C {
  int i;
}
```

The above has the same effect as:

```
namespace A {
  inline namespace B {
    namespace C {
      int i;
    }
  }
}
```

— *end example*]